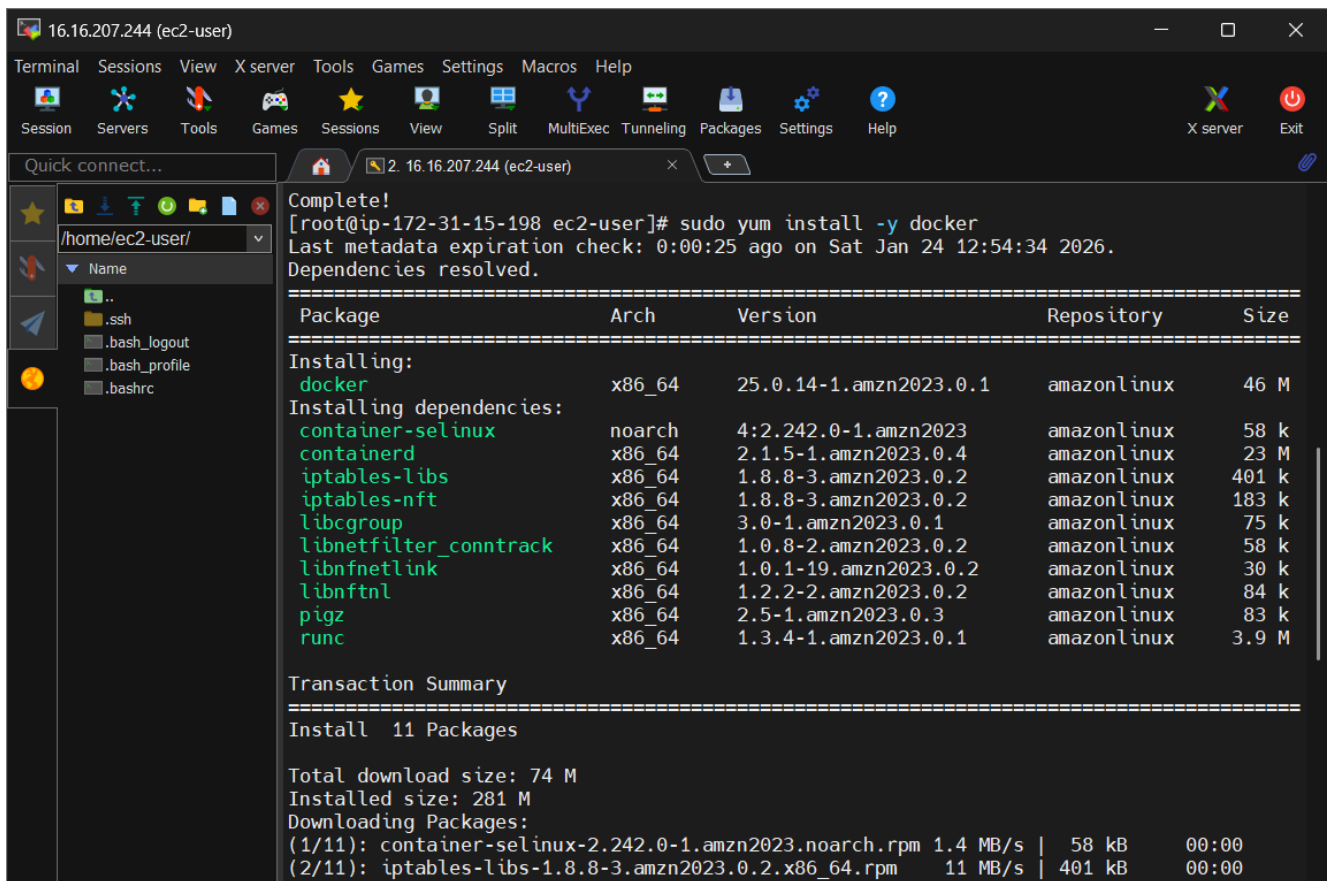


# Deploying HTML app - Docker

Name: Pooja Prakash Belgaumkar

1. First, we create an **EC2 instance** and connect to it using **MobaXterm**.
2. Then, we install **Docker** on the server to create and manage containers.
3. After that, we install **Git** in order to clone the project repository from **GitHub**.



The screenshot shows a MobaXterm terminal window connected to an EC2 instance with IP 16.16.207.244. The terminal displays the command to install Docker and its dependencies, followed by a detailed list of installed packages and their sizes.

```
Complete!
[root@ip-172-31-15-198 ec2-user]# sudo yum install -y docker
Last metadata expiration check: 0:00:25 ago on Sat Jan 24 12:54:34 2026.
Dependencies resolved.
=====
Package                Arch      Version                               Repository    Size
=====
Installing:
docker                  x86_64    25.0.14-1.amzn2023.0.1              amazonlinux   46 M
Installing dependencies:
container-selinux       noarch    4:2.242.0-1.amzn2023                amazonlinux   58 k
containerd              x86_64    2.1.5-1.amzn2023.0.4                amazonlinux   23 M
iptables-libs           x86_64    1.8.8-3.amzn2023.0.2                amazonlinux   401 k
iptables-nft            x86_64    1.8.8-3.amzn2023.0.2                amazonlinux   183 k
libcgroup              x86_64    3.0-1.amzn2023.0.1                  amazonlinux   75 k
libnetfilter_conntrack x86_64    1.0.8-2.amzn2023.0.2                amazonlinux   58 k
libnftnl                x86_64    1.0.1-19.amzn2023.0.2               amazonlinux   30 k
libnftnl                x86_64    1.2.2-2.amzn2023.0.2                amazonlinux   84 k
pigz                    x86_64    2.5-1.amzn2023.0.3                  amazonlinux   83 k
runc                    x86_64    1.3.4-1.amzn2023.0.1                amazonlinux   3.9 M
=====

Transaction Summary
=====
Install 11 Packages

Total download size: 74 M
Installed size: 281 M
Downloading Packages:
(1/11): container-selinux-2.242.0-1.amzn2023.noarch.rpm 1.4 MB/s | 58 kB 00:00
(2/11): iptables-libs-1.8.8-3.amzn2023.0.2.x86_64.rpm 11 MB/s | 401 kB 00:00
(3/11): iptables-nft-1.8.8-3.amzn2023.0.2.x86_64.rpm 5.6 MB/s | 183 kB 00:00
(4/11): containerd-2.1.5-1.amzn2023.0.4.x86_64.rpm 1.4 MB/s | 23 MB 00:00
(5/11): docker-25.0.14-1.amzn2023.0.1.x86_64.rpm 1.4 MB/s | 46 MB 00:00
(6/11): libnftnl-1.0.1-19.amzn2023.0.2.x86_64.rpm 1.4 MB/s | 30 kB 00:00
(7/11): libnftnl-1.2.2-2.amzn2023.0.2.x86_64.rpm 1.4 MB/s | 84 kB 00:00
(8/11): libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64.rpm 1.4 MB/s | 58 kB 00:00
(9/11): libcgroup-3.0-1.amzn2023.0.1.x86_64.rpm 1.4 MB/s | 75 kB 00:00
(10/11): pigz-2.5-1.amzn2023.0.3.x86_64.rpm 1.4 MB/s | 83 kB 00:00
(11/11): runc-1.3.4-1.amzn2023.0.1.x86_64.rpm 1.4 MB/s | 3.9 MB 00:00
=====
```

```
[root@ip-172-31-15-198 ec2-user]# sudo systemctl enable docker.service
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
[root@ip-172-31-15-198 ec2-user]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-15-198 ec2-user]# yum install git -y
Last metadata expiration check: 0:25:36 ago on Sat Jan 24 12:54:34 2026.
Dependencies resolved.
```

| Package                         | Arch   | Version                  | Repository  | Size  |
|---------------------------------|--------|--------------------------|-------------|-------|
| <b>Installing:</b>              |        |                          |             |       |
| git                             | x86_64 | 2.50.1-1.amzn2023.0.1    | amazonlinux | 53 k  |
| <b>Installing dependencies:</b> |        |                          |             |       |
| git-core                        | x86_64 | 2.50.1-1.amzn2023.0.1    | amazonlinux | 4.9 M |
| git-core-doc                    | noarch | 2.50.1-1.amzn2023.0.1    | amazonlinux | 2.8 M |
| perl-Error                      | noarch | 1:0.17029-5.amzn2023.0.2 | amazonlinux | 41 k  |
| perl-File-Find                  | noarch | 1.37-477.amzn2023.0.7    | amazonlinux | 25 k  |
| perl-Git                        | noarch | 2.50.1-1.amzn2023.0.1    | amazonlinux | 41 k  |
| perl-TermReadKey                | x86_64 | 2.38-9.amzn2023.0.2      | amazonlinux | 36 k  |
| perl-lib                        | x86_64 | 0.65-477.amzn2023.0.7    | amazonlinux | 15 k  |

- Next, we clone the project from GitHub and change the pwd to **ECR-ECS-Project**, where all the required application files are present.

```
[root@ip-172-31-15-198 ec2-user]# git clone https://github.com/PoojaPrakash08/ECR-ECS-Project.git
Cloning into 'ECR-ECS-Project'...
remote: Enumerating objects: 122, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 122 (delta 16), reused 5 (delta 5), pack-reused 101 (from 5)
Receiving objects: 100% (122/122), 1.27 MiB | 15.65 MiB/s, done.
Resolving deltas: 100% (50/50), done.
[root@ip-172-31-15-198 ec2-user]# ls
ECR-ECS-Project
[root@ip-172-31-15-198 ec2-user]# cd ECR-ECS-Project/
[root@ip-172-31-15-198 ECR-ECS-Project]# ls
Dockerfile      buildspec.yml  fonts           index.html
about.html      contact.html   guard.html      js
'buildspec (1).yaml'  css           images          service.html
[root@ip-172-31-15-198 ECR-ECS-Project]#
```

- Then, we create a **Dockerfile** to build a Docker image for the HTML application.

- In the Dockerfile, we install an **HTML web server** such as **Apache** or **Nginx**.

- The default directory to store HTML files is `/var/www/html/` for Apache and `/usr/share/nginx/html/` for nginx.
- The application is configured to run on **port 80** inside the container.

#### 1. DockerFile type 1

```
FROM ubuntu
RUN apt update -y
RUN apt install apache2 -y
RUN apt install apache2-utils -y
COPY ECR-ECS-Project//var/www/html/
RUN service apache2 restart
EXPOSE 80
CMD[ "/usr/sbin/apachectl", "-D", "FOREGROUND" ]
```

#### 1. DockerFile type 2

```
FROM nginx:latest
COPY . /usr/share/nginx/html/
EXPOSE 80
```

### 6. After writing the Dockerfile, we save it inside the **application directory**

```
[root@ip-172-31-15-198 ec2-user]# ls
ECR-ECS-Project
[root@ip-172-31-15-198 ec2-user]# cd ECR-ECS-Project/
[root@ip-172-31-15-198 ECR-ECS-Project]# ls
Dockerfile      buildspec.yml  fonts          index.html
about.html      contact.html   guard.html     js
'buildspec (1).yaml'  css          images         service.html
[root@ip-172-31-15-198 ECR-ECS-Project]# cat Dockerfile
FROM nginx:latest

COPY . /usr/share/nginx/html/

EXPOSE 80
[root@ip-172-31-15-198 ECR-ECS-Project]# docker images
REPOSITORY    TAG       IMAGE ID   CREATED   SIZE
```

7. Then, we create a Docker image named **my-app:v1** using the Dockerfile.

```
[root@ip-172-31-15-198 ECR-ECS-Project]# docker build -t my-app:v1 .
[+] Building 5.8s (7/7) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 156B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:latest  1.8s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.1s
=> => transferring context: 3.16MB                               0.0s
=> [1/2] FROM docker.io/library/nginx:latest@sha256:c881927c4077710ac4b1da63b83a 3.7s
=> => resolve docker.io/library/nginx:latest@sha256:c881927c4077710ac4b1da63b83a 0.0s
=> => sha256:617fef3c6adb90788d06f7ebe634fe990a6cd1e63c815facaf5 2.29kB / 2.29kB 0.0s
=> => sha256:4af177a024eb8a1e43f4fb6c66735bb8260115cb5925a64f516 9.09kB / 9.09kB 0.0s
=> => sha256:119d43eec815e5f9a47da3a7d59454581b1e204b0c34db86f 29.77MB / 29.77MB 0.8s
=> => sha256:700146c8ad64762607eb0076d9fe376bed85c65f27650a4f3 33.07MB / 33.07MB 1.2s
=> => sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267 10.23kB / 10.23kB 0.0s
=> => sha256:d989100b8a84afca8cb4b5bcc62beec741cb69e181fe7815d79e4aa 628B / 628B 0.6s
=> => sha256:500799c304244b0545f8f1d54d86d3a17512c219cd0edd3d5f5b60f 955B / 955B 0.9s
=> => extracting sha256:119d43eec815e5f9a47da3a7d59454581b1e204b0c34db86f171b7ce 1.3s
=> => sha256:10b68cfefee1d8726f1f754e9f566b73bd4df2531476315e6ac55b4 404B / 404B 1.1s
=> => sha256:57f0dd1befe210f3a7186fe46ad1552d4bc8864701c8cec5806 1.21kB / 1.21kB 1.2s
=> => sha256:eaf8753feae0b5aaadb86ac2cb972ff57cff12f877ad50c548f 1.40kB / 1.40kB 1.4s
=> => extracting sha256:700146c8ad64762607eb0076d9fe376bed85c65f27650a4f3b76fc66 1.1s
=> => extracting sha256:d989100b8a84afca8cb4b5bcc62beec741cb69e181fe7815d79e4aa0 0.0s
=> => extracting sha256:500799c304244b0545f8f1d54d86d3a17512c219cd0edd3d5f5b60f6 0.0s
=> => extracting sha256:10b68cfefee1d8726f1f754e9f566b73bd4df2531476315e6ac55b4f 0.0s
=> => extracting sha256:57f0dd1befe210f3a7186fe46ad1552d4bc8864701c8cec580676144 0.0s
=> => extracting sha256:eaf8753feae0b5aaadb86ac2cb972ff57cff12f877ad50c548fde809 0.0s
=> [2/2] COPY . /usr/share/nginx/html/                          0.2s
=> exporting to image                                           0.1s
=> => exporting layers                                           0.0s
=> => writing image sha256:9fd78fdbb4219b93c15f895c7bd97b47fa93e2e1819eaea91146a 0.0s
=> => naming to docker.io/library/my-app:v1                     0.0s
[root@ip-172-31-15-198 ECR-ECS-Project]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
my-app v1 9fd78fdbb421 About a minute ago 164MB
[root@ip-172-31-15-198 ECR-ECS-Project]#
```

```
[root@ip-172-31-15-198 ECR-ECS-Project]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
my-app v1 9fd78fdbb421 About a minute ago 164MB
[root@ip-172-31-15-198 ECR-ECS-Project]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS NAMES
549133eb7777e244310b64478b3e9deda01e79df1e1d8cab108c473daddb957a my-app:v1 "/docker-entrypoint..." 6 seconds ago Up 5 seconds 80/tcp, 0.0.0.0:80->90/tcp, :::80->90/tcp myapp-container
[root@ip-172-31-15-198 ECR-ECS-Project]#
```

- Using this Docker image, we create a Docker container named **myapp-container-2** to deploy the **ECR-ECS-Project** application.
- While running the container, we map **host port 8081** to **container port 80**, so the application can be accessed externally.

```
[root@ip-172-31-15-198 ECR-ECS-Project]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
[root@ip-172-31-15-198 ECR-ECS-Project]# docker run -itd --name myapp-container-2 -p 8081:80 my-app:v1
bd899783a3e8eb22f8b711e3c35bca50b12a9f41f273b2cf8f0e866e2794cc11
[root@ip-172-31-15-198 ECR-ECS-Project]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
bd899783a3e8   my-app:v1      "/docker-entripoint..." 4 seconds ago  Up 3 seconds  0.0.0.0:8081->80/tcp, :::8081->80/tcp  myapp-container-2
[root@ip-172-31-15-198 ECR-ECS-Project]#
```

- To allow access to the application, we add **inbound rules** in the EC2 security group for **port 8081** on the host.

EC2 > Security Groups > sg-0db2eed90a7b53d7e - launch-wizard-17 > Edit inbound rules

### Edit inbound rules

Inbound rules control the incoming traffic that's allowed to reach the instance.

| Security group rule ID | Type       | Protocol | Port range | Source   | Description - optional |        |
|------------------------|------------|----------|------------|----------|------------------------|--------|
| sgr-0446f8aae85d969cc  | SSH        | TCP      | 22         | Custom   | 0.0.0.0/0              | Delete |
| -                      | Custom TCP | TCP      | 80         | Anywh... | 0.0.0.0/0              | Delete |
| -                      | Custom TCP | TCP      | 8081       | Anywh... | 0.0.0.0/0              | Delete |

[Add rule](#)

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

[Cancel](#) [Preview changes](#) [Save rules](#)

11. Finally, when we access the application using the **public IP address of the EC2 instance along with port 8081**, the deployed HTML application running inside the Docker container is displayed.

